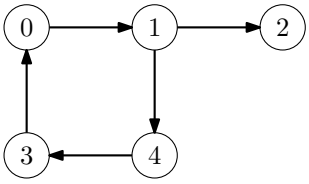
Algorithms for Data Science
Exercise 6

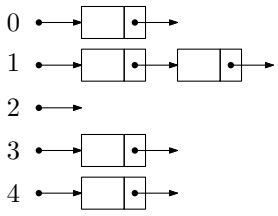
See the cheatsheet for valuable remarks!

Task 6.1: *(Simple Tasks) Graphs, Stack and BFS*

(a) Given the following directed graph, write the corresponding adjacency matrix and adjacency lists.



Adjacency lists:



Adjacency matrix:

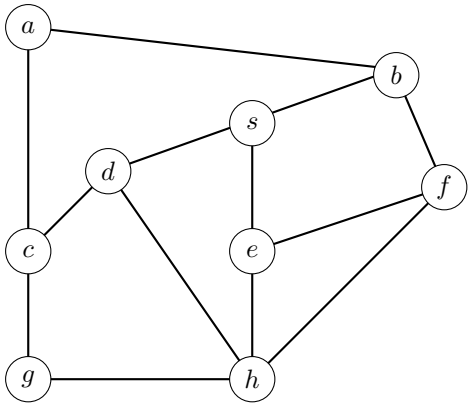
	0	1	2	3	4
0					
1					
2					
3					
4					

(b) Write the outputs of the following sequence of operations when the underlying data structure is an empty Stack.

`Add(0)`, `Add(4)`, `FindNewest()`, `Add(3)`, `ExtractNewest()`, `Add(2)`, `ExtractNewest()`, `Add(1)`, `ExtractNewest()`, `ExtractNewest()`, `FindNewest()`, `Add(2)`, `ExtractNewest()`, `ExtractNewest()`

(c) Breadth-first Search (BFS) is run on the following graph with starting node s. Order the vertices by their order of being extracted from the queue and compute their distances and parents. (If a vertex has more neighbors, we consider them in alphabetical order.)

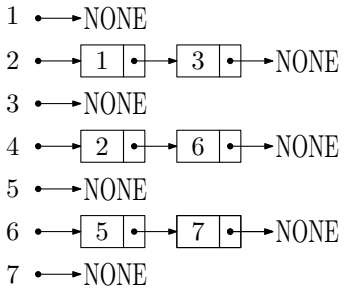
```
dist[s] = 0
parent[s] =NONE
q=new queue
q.add(s)
while q not empty do
    v = q.extractOldest()
    for every edge (v, w) do
        if dist[w] not computed yet
         then
            dist[w] = dist[v] + 1
            parent[w] = v
            q.add(w)
```



	s	a	b	c	d	e	f	g	h
distance									
parent									
removal order									

Task 6.2: *Drawing Graphs*

(a) Draw a directed graph without edge crossings corresponding to the following adjacency lists.



(b) Draw a (undirected) graph without edge crossings corresponding to the following adjacency matrix.

	1	2	3	4	5	6
1	0	1	0	1	0	1
2	1	0	0	0	1	0
3	0	0	0	1	0	1
4	1	0	1	0	1	0
5	0	1	0	1	0	1
6	1	0	1	0	1	0

Task 6.3: *Waiting in the Queue!*

Write the outputs of the following sequence of operations when the underlying data structure is an empty Queue.

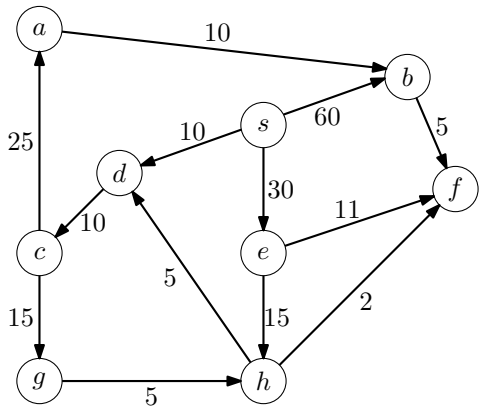
`Add(0)`, `Add(4)`, `FindOldest()`, `Add(3)`, `ExtractOldest()`, `Add(2)`, `ExtractOldest()`, `Add(1)`, `ExtractOldest()`, `ExtractOldest()`, `FindOldest()`, `Add(2)`, `ExtractOldest()`, `ExtractOldest()`

Task 6.4: *Dijkstra*

Dijkstra's algorithm (see below) is run on the following graph with starting node s. Compute the distances and parents of all nodes. Also mark all edges that belong to cheapest paths from s.

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time
     then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w}
             then
                dist[w] = dist[v]+c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



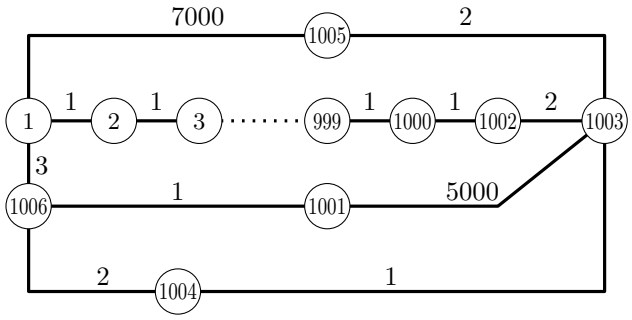
	s	a	b	c	d	e	f	g	h
distance									
parent									

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 6.5: *Incomplete Floyd-Warshall*

Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=1006 and the edge weights are given as below (the edges on the path from 3 to 999 have weight 1).



Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 9) already after 1002 iterations.

```
1  δ[1..1006][1..1006] = new 2D array filled with +∞
2  for i = 1 to 1006 do
3  |  δ[i][i] = 0
4  foreach edge (v, w, c) do
   |  // v and w are the endpoints and c is the weight of the edge
5  |  δ[v][w] = c
6  |  δ[w][v] = c
7  for k = 1 to 1002 do
8  |  foreach pair (v, w) do
9  |  |  δ[v][w] = min(δ[v][w],    δ[v][k] + δ[k][w])
10 return δ
```

Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 1002 times**!

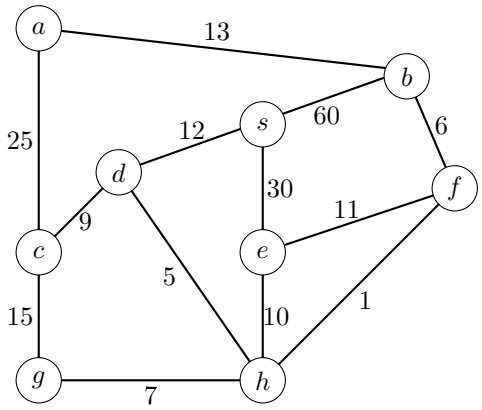
$\delta[1][1002] =$ _____ $\delta[1][1001] =$ _____ $\delta[1][1005] =$ _____

$\delta[2][4] =$ _____ $\delta[1003][1006] =$ _____ $\delta[1001][1005] =$ _____

$\delta[1003][1004] =$ _____ $\delta[1001][1004] =$ _____

Task 6.6: *Minimum Spanning Tree*

(a) Draw a minimum spanning tree in the following graph.



(b) A minimal spanning subgraph is always a tree if the edge weights are positive. Draw a weighted graph that has positive and negative edge weights and whose minimal spanning subgraph is not a tree.

Task 6.7: *Salt and Pepper*

For cooking one needs only salt and pepper. Among a group of n people, give every person either a salt shaker or a pepper shaker such that when any two friends meet, they can cook together. If such a distribution is not possible, say so. In total there are k pairs of friends. (It is possible that one person has several friends in the group.)

Model the problem as a graph problem and use (or modify) an appropriate graph algorithm from the lecture to solve it. Your solution should work in time $O(n + k)$.

(a) First, assume that your graph is connected!

(b) Next, give an algorithm that also works if the graph is not connected! (*Advanced task*: is your running time really $O(n + k)$?)

Task 6.8: *Is it true . . . ? (Graphs)*

(a) Is it true that any connected graph with $n - 1$ edges has no cycles? Prove the claim or draw a counter example.

(b) Also in this task, we consider only graphs without parallel edges. What is the maximum number of edges . . .

i. . . . in a directed graph when we allow self-loop edges?

ii. . . . in a directed graph when we do not allow self-loop edges?

iii. . . . in an (undirected) graph when we do not allow self-loop edges?

Task 6.9: *Floyd-Warshall vs Dijkstra*

In the lecture, we have seen that the problem of computing the cheapest path from every vertex to every other can be solved via the Floyd-Warshall algorithm in time $\Theta(n^3 + m)$ or by running Dijkstra's algorithm from every vertex in total time $\Theta(mn \log m + n^2)$ (on the slides we further simplified these running times assuming that there are no parallel edges).

What are the running times of the two approaches if . . .

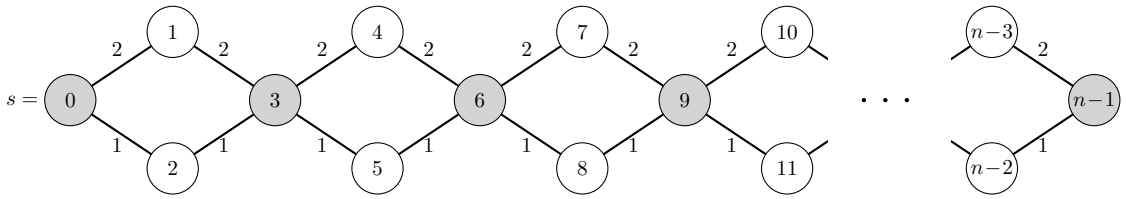
(a) . . . m is extremely small, that is, $m = \Theta(1)$? Give the running times in dependence of only n.

(b) . . . m is extremely large, that is, $m = \Theta(n^4)$ (thus, we have parallel edges)? Give the running times in dependence of only m.

Task 6.10: *Dijkstra with a Normal Queue*

In this task, you learn why it is very important to use a priority queue in Dijkstra's algorithm.

Consider the following graph. Argue that, in the worst case, Dijkstra's algorithm will run immensenly much slower on that graph if we use a normal queue (as in the algorithm below) instead of a priority queue!



$dist[0..n-1]$ filled with INF
$dist[s] = 0$
q=new queue
q.`add(`s`)`
while *q not empty* **do**
  $v = q$.`removeOldest()`
  if *v removed for the first time* **then**
    for *every edge* (v, w) **do**
      if $dist[w] > dist[v] + c_{v,w}$ **then**
        $dist[w] = dist[v] + c_{v,w}$
        q.`add(`w`)`

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 6.11: *Output the Cheapest Paths Using Floyd-Warshall*

(a) Extend the algorithm of Floyd-Warshall such that it outputs the actual cheapest paths in additional time $O(n)$ for each pair.

Note: Print the vertices as they appear on the path, one by one.

(b) Now suppose that you are only given the δ table as computed by Floyd Warshall and that you don't have access to the graph. Give an $O(n^2)$-time algorithm that, given a pair of vertices u and v, reconstructs a cheapest path from u and v. Assume that the δ table already lies in the memory and that it takes $O(1)$ time to read a single entry of the table.

Note: Print the edges as they appear on the path, one by one.